Heaps, Stacks, Sorting and Binary Search

4 questions to study here, each on its own page, with the full interview answer and code

15 more fit this topic but are covered in another topic: they are listed as SKIP at the end of the contents

1	3
HARD	MEDIUM

How to use

- Questions are Hard first, then Medium, then Easy, newest last-asked date first.
- A question that fits several topics has its pages in only one PDF; the other topics list it as SKIP and name where it is.
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Heaps, Stacks, Sorting and Binary Search | 4 questions | tap a title to jump to it

H 1. Median of Two Sorted Arrays (LC 4) with a Follow-up Variant **2026-10-09**

M 2. Streaming Median (LC 295) with Follow-ups **2026-08-03**

M 3. Top-K Frequent in Large Logs **2026-07-19**

M 4. First Bad Version with Parallel Search Follow-up **2026-06-15**

SKIP: these fit this topic but have their pages in another topic (15)

H SKIP Huffman Tree Construction **topic 5: Trees and Tries**

M SKIP Bounded Knapsack Variant **topic 8: Greedy and Math**

M SKIP Visible People in a Line (Last Person, Then Everyone) **topic 1: Arrays, Hashing and Two Pointers**

M SKIP Path Minimizing the Maximum Cell (Swim in Rising Water) **topic 4: Graphs and Search**

M SKIP Meeting Rooms / Interval Scheduling Variants **topic 1: Arrays, Hashing and Two Pointers**

M SKIP Active Communication Top Users **topic 1: Arrays, Hashing and Two Pointers**

M SKIP Implement Python's itertools.tee **topic 9: Design, Streaming and Scale**

M SKIP Union of Sorted Interval Lists **topic 1: Arrays, Hashing and Two Pointers**

M SKIP Cutting Ribbons (LC 1891) **topic 1: Arrays, Hashing and Two Pointers**

M SKIP Progressive Array Partitioning **topic 1: Arrays, Hashing and Two Pointers**

M SKIP Decode String **topic 2: Strings, Parsing and Recursion**

M SKIP Batch and Streaming Log Deduplication **topic 9: Design, Streaming and Scale**

M SKIP Streaming Points: Emit K Within Distance **topic 9: Design, Streaming and Scale**

M SKIP Nested Function Expression Evaluator **topic 2: Strings, Parsing and Recursion**

M SKIP Russian Doll Envelopes **topic 1: Arrays, Hashing and Two Pointers**

STUDY HERE: topic 7, Heaps, Stacks, Sorting and Binary Search

Site tags: binary-search, heap, divide-and-conquer, hard

HARD | IN APP LC 4 | 1Point3Acres | tech-screen | 1 of 4

Median of Two Sorted Arrays (LC 4) with a Follow-up Variant

ASKED 2026-10-09 | 11 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | ALL TIME 74.2 | 30 DAYS 61 | PRIORITY 4/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks, Sorting and Binary Search

Optimal answer from: your app's interview approach (LeetCode #4)

LeetCode match: LC 4 Median of Two Sorted Arrays

Frequency: Single report | Last asked: 2026-10-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/median-two-sorted-arrays-k-inputs

QUESTION

L5 R1 coding: LC 4 (Median of Two Sorted Arrays). Heading toward the LC 295 streaming two-heap approach drew a redirect from the interviewer. Follow-up: extend to many sorted inputs, in the spirit of LC 23 (Merge k Sorted Lists).

Per the candidate, the base question is LC 4 (Median of Two Sorted Arrays): given two sorted arrays, return the median of their combined elements.
Follow-up: the input becomes many sorted arrays instead of two, in the spirit of LC 23 (Merge k Sorted Lists).

APPROACH

Drill LC 4 until you can explain the partition invariant and the odd/even median cases without notes.
Drill LC 23 with a min-heap and with divide-and-conquer merging, and compare their costs as the number of inputs grows.
Practice stating, in under a minute, why a streaming two-heap approach is unnecessary when the inputs are static and sorted.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the median definition when the total is even (the mean of the two middle values), whether the arrays can be empty, whether duplicates are possible, and the sizes.

BRUTE FORCE TO SAY OUT LOUD

Merge and sort, then read the middle. O((m + n) log(m + n)); merging two sorted arrays with two pointers is O(m + n).

OPTIMAL IDEA

Binary search the cut in the shorter array: choose i in a and j = (m + n + 1) // 2 - i in b so the left parts hold half of the elements. The cut is valid when a[i-1] <= b[j] and b[j-1] <= a[i], with minus and plus infinity at the ends. The answer is the larger left value for an odd total, or the mean of the larger left and the smaller right value. O(log min(m, n)). For k arrays: binary search on the value and count elements at most x with a bisect per array, O(k log n log range), or pop from a min-heap until the middle, O(N log k).

DRY-RUN THESE CASES

One array empty, both of equal length, total odd and even, all elements equal, one array entirely smaller than the other, negative numbers.

LIKELY FOLLOW-UPS

Many sorted arrays (the stated follow-up), a stream (two heaps), or the k-th element instead of the median.

PITFALLS

Starting from the two-heap design of the streaming median (wrong family for static sorted input). Searching the longer array and running past its end. Off-by-one in the half size for odd totals.

HOW TO THINK ABOUT IT

Both inputs are already sorted, so the median is a partition problem, not a streaming one (the two-heap design of LeetCode 295 is the wrong family and the interviewer redirects it). Split the small array at i and the large one at j so that the left halves together hold half the elements; the split is right when every left value is at most every right value, which only needs two comparisons across the cut. That condition is monotone in i, so binary search it in O(log min(m, n)). For k arrays the same idea becomes: binary search on the value and count elements at most x with one bisect per array.

Basis: LeetCode 4, Median of Two Sorted Arrays; the follow-up to many sorted inputs is in the spirit of LeetCode 23, Merge k Sorted Lists.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def median(a, b):
    merged = sorted(a + b)
    n, mid = len(merged), len(merged) // 2
    if n % 2:
        return merged[mid]
    return (merged[mid - 1] + merged[mid]) / 2
```

TIME	SPACE
O((m + n) log(m + n))	O(m + n)

WHY IT IS SLOW

Sorting ignores that the inputs are sorted and costs O((m + n) log(m + n)) time and O(m + n) memory. A binary search for the partition reads only O(log min(m, n)) values.

OPTIMAL SOLUTION · Binary search on the partition

TIME	SPACE
O(log min(m, n))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Now there are k sorted arrays instead of two. How do you find the median?

Binary search on the answer value. For a candidate x, count the elements at most x with one bisect per array (O(k log n)); the median is the smallest x whose count reaches the middle rank. With integers the search range is the global min to max, so the cost is O(k log n log(range)). The heap alternative pops the smallest element from a k-way merge until it passes the middle, O((N / 2) log k), which is simpler but linear in the total size.

FOLLOW-UP 2 What is the heap version and when would you choose it?

A min-heap holds the front of every array. Pop (N + 1) / 2 times, remembering the one or two middle values. It is O(N log k) at worst, trivial to get right and needs no bound on the values, so choose it when the arrays are small or the values are not integers; choose the value search when N is huge and k is small.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import bisect
import heapq

INF = float("inf")


# BASE QUESTION (optimal): Binary search on the partition
def median_two(a, b):
    """Both arrays are sorted: binary search the cut. The
    left parts must hold half of all the elements."""
    if len(a) > len(b):
        a, b = b, a
    m, n = len(a), len(b)
    if m + n == 0:
        raise ValueError("no elements")
    lo, hi, half = 0, m, (m + n + 1) // 2
    while lo <= hi:
        i = (lo + hi) // 2
        j = half - i
        # values just left and right of each cut
        a_left = a[i - 1] if i > 0 else -INF
        a_right = a[i] if i < m else INF
        b_left = b[j - 1] if j > 0 else -INF
        b_right = b[j] if j < n else INF
        if a_left <= b_right and b_left <= a_right:
            # a valid cut: read the median off its edges
            if (m + n) % 2:
                return max(a_left, b_left)
            return (max(a_left, b_left)
                    + min(a_right, b_right)) / 2
        if a_left > b_right:
            hi = i - 1
        else:
            lo = i + 1


# FOLLOW-UP 1: Now there are k sorted arrays instead of
# two. How do you find the ...
def median_k_value(arrays):
    """k arrays: binary search the value; one bisect per
    array counts how many elements are at most x."""
    arrays = [a for a in arrays if a]
    total = sum(len(a) for a in arrays)
    if total == 0:
        raise ValueError("no elements")

    def kth(rank):
        # smallest x with at least `rank` elements <= x
        lo = min(a[0] for a in arrays)
        hi = max(a[-1] for a in arrays)
        while lo < hi:
            mid = (lo + hi) // 2
            count = sum(bisect.bisect_right(a, mid)
                        for a in arrays)
            if count >= rank:
                hi = mid
            else:
                lo = mid + 1
        return lo

    if total % 2:
        return kth(total // 2 + 1)
    return (kth(total // 2) + kth(total // 2 + 1)) / 2


# FOLLOW-UP 2: What is the heap version and when would you
# choose it?
def median_k_heap(arrays):
    """k arrays, simple version: pop from a k-way merge
    until the middle one or two values have been seen."""
    heap = [(a[0], i, 0)
            for i, a in enumerate(arrays) if a]
    heapq.heapify(heap)
    total = sum(len(a) for a in arrays)
    if total == 0:
        raise ValueError("no elements")
    low_rank, high_rank = (total - 1) // 2, total // 2
    seen, middle = 0, []
    while heap:
        value, i, j = heapq.heappop(heap)
        if seen >= low_rank:
            middle.append(value)
        if seen >= high_rank:
            break
        seen += 1
        if j + 1 < len(arrays[i]):
            nxt = (arrays[i][j + 1], i, j + 1)
            heapq.heappush(heap, nxt)
    if total % 2:
        return middle[0]
    return (middle[0] + middle[1]) / 2


# CHECKER (brute force)
def median_brute(*arrays):
    # the oracle: merge, sort, read the middle
    merged = sorted(x for a in arrays for x in a)
    n, mid = len(merged), len(merged) // 2
```

```
    if n % 2:
        return merged[mid]
    return (merged[mid - 1] + merged[mid]) / 2


if __name__ == "__main__":
    assert median_two([1, 3], [2]) == 2
    assert median_two([1, 2], [3, 4]) == 2.5
    assert median_two([], [5]) == 5
    assert median_k_value([[1, 3], [2], [4, 5, 6]]) == 3.5
    assert median_k_heap([[1, 3], [2], [4, 5, 6]]) == 3.5
    import random
    for _ in range(300):
        arrays = []
        for _ in range(random.randint(1, 5)):
            size = random.randint(0, 6)
            arrays.append(sorted(random.randint(-9, 9)
                                 for _ in range(size)))
        if not any(arrays):
            continue
        want = median_brute(*arrays)
        assert median_k_value(arrays) == want
        assert median_k_heap(arrays) == want
        if len(arrays) >= 2:
            rest = sum(arrays[1:], [])
            rest.sort()
            assert median_two(arrays[0], rest) == want
```

NOTES FROM CANDIDATES

Pitfalls: Starting from the LC 295 (Find Median from Data Stream) two-heap design is the wrong family for the base question; the interviewer stepped in with a hint. Both inputs are already sorted — lean on that.

Follow-ups: Many sorted inputs instead of two

PREPARATION

Drill LC 4 until you can explain the partition invariant and the odd/even median cases without notes.
Drill LC 23 with a min-heap and with divide-and-conquer merging, and compare their costs as the number of inputs grows.
Practice stating, in under a minute, why a streaming two-heap approach is unnecessary when the inputs are static and sorted.

STUDY HERE: topic 7, Heaps, Stacks, Sorting and Binary Search

Also fits: Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: heap, streaming, medium

MEDIUM | IN APP LC 295 | 1Point3Acres | tech-screen | 2 of 4

Streaming Median (LC 295) with Follow-ups

ASKED 2026-08-03 | 78 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 50.4 | 30 DAYS 0 | PRIORITY 69/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks, Sorting and Binary Search

Optimal answer from: your app's interview approach (LeetCode #295)

LeetCode match: LC 295 Find Median from Data Stream

Frequency: Single report | Last asked: 2026-08-03 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/streaming-median-with-follow-ups

QUESTION

A near-canonical LC 295 streaming-median implementation using two heaps, followed by two design extensions: exploit a very small value domain, then support an arbitrary k-th-smallest query instead of only the median.SWEheapstreamingmediumFrequencySingle reportLast asked2026-08-03Stagetech-screen

APPROACH

Implement the two-heap median structure and dry-run odd and even stream lengths.
Sketch a bounded-domain alternative and state its update and query costs.
Compare what must change when the requested order statistic is an arbitrary k rather than a fixed median.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether values are integers, the range of values, how often median queries happen compared with inserts, and what to return for an even count (average of two middles).

BRUTE FORCE TO SAY OUT LOUD

Keep a sorted list and insert with bisect; median is the middle. O(n) per insert due to shifting.

OPTIMAL IDEA

Two heaps: a max-heap for the lower half and a min-heap for the upper half, sizes within one. Insert by pushing through one heap into the other and rebalancing. Median is the top of the larger heap or the average of both tops. O(log n) insert, O(1) median.

DRY-RUN THESE CASES

One element, two elements, duplicates, all equal, alternating very large and very small values.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

Values in a small range (counting array), arbitrary k-th smallest (Fenwick tree or order-statistic tree), or too much data (quantile sketch).

PITFALLS

Rebalancing after the wrong heap push. Integer division when the median should be fractional.

HOW TO THINK ABOUT IT

Keep the data split at the median: a max-heap for the smaller half and a min-heap for the larger half, with sizes equal or differing by one. Then the median is read from the tops. Insert by pushing through one heap into the other to keep every smaller value on the left, and rebalance the sizes.

Basis: LeetCode 295, Find Median from Data Stream.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
nums = []  # always kept sorted
def add(x):
    insort(nums, x)
def median():
    n = len(nums)
    if n % 2:
        return nums[n // 2]
    return (nums[n // 2 - 1] + nums[n // 2]) / 2
```

TIME	SPACE
O(n) per insert (list shift), O(1) median	O(n)

WHY IT IS SLOW

Each insert shifts up to n elements, O(n) per add (fast in practice, but linear). Two heaps keep the median available with O(log n) per add and O(1) per query.

OPTIMAL SOLUTION · Two heaps

TIME	SPACE
O(log n) add, O(1) median	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Almost every value is in a small range (say 0 to 100). Can you do better than two heaps?

Yes: a counting array gives O(1) inserts, and the median needs one pass over at most V counts, so O(V) per query with V fixed. Keep a moving pointer to the median bucket to make queries amortized O(1).

FOLLOW-UP 2 Now support an arbitrary k-th smallest, not only the median.

Two heaps only work for one fixed rank. Use an order-statistics structure: a Fenwick tree over (compressed) values, or a balanced BST augmented with subtree sizes. Both give O(log n) add and O(log n) k-th, and the Fenwick tree is far shorter to write in an interview.

FOLLOW-UP 3 What if the data stream is too large to store?

Exact medians need memory proportional to the data, so use a quantile sketch (t-digest, GK, KLL). They answer any quantile within a chosen error bound in O(1/epsilon) memory.

FOLLOW-UP 4 The median of only the last K values.

Keep the window in a deque (to know what expires) and a sorted list (to read the middle); insert and remove with bisect. O(K) per update because of list shifting; two heaps with lazy deletion give O(log K).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
import heapq
from bisect import bisect_left, insort


# BASE QUESTION (optimal): Two heaps
class MedianFinder:
    def __init__(self):
        # low: max-heap (stored negated); high: min-heap
        self.low, self.high = [], []

    def add(self, x):
        # push through low so the halves stay ordered
        heapq.heappush(self.low, -x)
        top = -heapq.heappop(self.low)
        heapq.heappush(self.high, top)
        # keep low the same size as high, or one bigger
        if len(self.high) > len(self.low):
            top = heapq.heappop(self.high)
            heapq.heappush(self.low, -top)

    def median(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2


# FOLLOW-UP 1: Almost every value is in a small range (say
# 0 to 100). Can you do ...
class CountingMedian:
    """Follow-up: values are whole numbers in 0..maxv."""

    def __init__(self, maxv):
        self.counts = [0] * (maxv + 1)
        self.n = 0

    def add(self, x):
        self.counts[x] += 1
        self.n += 1

    def kth(self, k):
        # k-th smallest (1-based): one pass over the
        # counts
        seen = 0
        for value, c in enumerate(self.counts):
            seen += c
            if seen >= k:
                return value

    def median(self):
        lo = self.kth((self.n + 1) // 2)
        hi = self.kth(self.n // 2 + 1)
        return (lo + hi) / 2


# FOLLOW-UP 2: Now support an arbitrary k-th smallest, not
# only the median.
class Fenwick:
    """Follow-up: any k-th smallest in O(log V)."""

    def __init__(self, size):
        self.size, self.tree = size, [0] * (size + 1)

    def add(self, value):
        i = value + 1
        while i <= self.size:
            self.tree[i] += 1
            i += i & -i

    def kth(self, k):
        # walk down the tree to the k-th smallest value
        pos, step = 0, 1 << self.size.bit_length()
        while step:
            nxt = pos + step
            if nxt <= self.size and self.tree[nxt] < k:
                pos = nxt
                k -= self.tree[nxt]
            step >>= 1
        return pos


# FOLLOW-UP 4: The median of only the last K values.
class WindowMedian:
    """Follow-up: median of only the last k values."""

    def __init__(self, k):
        self.k, self.order, self.sorted = k, [], []

    def add(self, x):
        self.order.append(x)
        insort(self.sorted, x)
        if len(self.order) > self.k:
            old = self.order.pop(0)
            del self.sorted[bisect_left(self.sorted, old)]

    def median(self):
        s, n = self.sorted, len(self.sorted)
        return (s[(n - 1) // 2] + s[n // 2]) / 2


if __name__ == "__main__":
    m = MedianFinder()
    for x in [5, 2, 8]:
        m.add(x)
    assert m.median() == 5
    m.add(1)
    assert m.median() == 3.5
    c = CountingMedian(100)
    for x in [5, 2, 8, 1]:
        c.add(x)
```

```
    assert c.median() == 3.5
    f = Fenwick(100)
    for x in [5, 2, 8, 1]:
        f.add(x)
    ranks = [f.kth(k) for k in (1, 2, 3, 4)]
    assert ranks == [1, 2, 5, 8]
    w = WindowMedian(3)
    for x in [9, 1, 5, 7]:
        w.add(x)
    # window is [1, 5, 7]
    assert w.median() == 5
```

NOTES FROM CANDIDATES

Keep the lower half in a max-heap and the upper half in a min-heap. Maintain a size difference of at most one and keep every lower-half value no greater than every upper-half value.
For an even-sized stream, average the two heap roots.
Each insertion costs O(log n), a median query costs O(1), and the heaps use O(n) space.
For a small bounded domain, a frequency array gives O(1) updates and an O(U) order-statistic scan, where U is the domain size. For arbitrary k-th-smallest queries, use an order-statistic tree or a Fenwick/segment tree over a known or compressed domain for logarithmic update and selection.
The base problem required complete working code plus a dry run and complexity analysis.
The first follow-up stopped at design and pseudocode; the second was discussion-only because the round ran out of time.

PREPARATION

Implement the two-heap median structure and dry-run odd and even stream lengths.
Sketch a bounded-domain alternative and state its update and query costs.
Compare what must change when the requested order statistic is an arbitrary k rather than a fixed median.

STUDY HERE: topic 7, Heaps, Stacks, Sorting and Binary Search

Also fits: Arrays, Hashing and Two Pointers, Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: heap, hashmap, scaling, medium

MEDIUM | IN APP LC 347 | 1Point3Acres | phone-screen, onsite-coding | 3 of 4

★ TOP 75: rank 29 in the Medium and Hard priority list

★ Top-K Frequent in Large Logs

ASKED 2026-07-19 | 93 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 58.8** | **30 DAYS 41.1** | **PRIORITY 21/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks, Sorting and Binary Search

Optimal answer from: your app's interview approach (LeetCode #347)

LeetCode match: LC 347 Top K Frequent Elements

Frequency: Medium | Last asked: 2026-07-19 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/top-k-frequent-from-logs

QUESTION

EXAMPLES

```
Input:  chatLog = ["A says hi", "B says hi", "A says
    bye", "C says hi", "A says yo"], k=2
Counts: A=3, B=1, C=1
Output: [A, B]  // or [A, C] — ties allowed
```

APPROACH

Practice writing quickselect with the partition step (and explain how to recover from a bad pivot).
Be able to state amortized vs worst-case time for each approach.
Have a one-line answer for streaming: "hashmap counts + heap of size K maintained online".
Drill the distributed answer once: per-shard top-K, then K-way merge of the partial heaps.

HOW TO THINK ABOUT IT

Count the speakers first; a log line is just a way to read the user id. Then you only need the K largest counts: a heap of size K does that without sorting all users. For logs spread over many machines the local top K are not enough on their own, so say what extra information you send.

Basis: LeetCode 347, Top K Frequent Elements (here the key is the speaker in each chat line).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(chat_log, k):
    names = [l.split()[0] for l in chat_log]
    counts = Counter(names)
    def rank(name):
        return (-counts[name], name)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(n + d log d)	O(d)

WHY IT IS SLOW

Sorts all d distinct speakers, O(d log d), when only K are wanted; a size-K heap is O(d log k), and a stream needs a counter sketch instead of exact counts.

OPTIMAL SOLUTION · Count, then keep a size-K heap

TIME	SPACE
O(n + d log k)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you do it across many machines when the log does not fit on one?

Count per shard and send each shard's local top-K to a coordinator. That is not exact in general (a user can be 11th everywhere yet first overall), so for an exact answer use the threshold algorithm: gather the local top-K, compute a global lower bound, ask every shard for all users above bound / numShards, and merge. If an approximate answer is acceptable, merge count-min sketches (they add) and keep a heap of the K largest estimates.

FOLLOW-UP 2 What if the log arrives as an unbounded stream?

Use the Misra-Gries or Space-Saving summary with m counters. Any user with more than n / (m + 1) messages is guaranteed to be in the summary, and a second pass (or a count-min sketch for the estimate) can confirm the real counts. **Cost:** O(1) per message with O(m) memory for m counters (Misra-Gries), instead of O(d) memory for the exact counts.

FOLLOW-UP 3 Which approach would you choose and why?

A heap is the safe default: simple, O(n + d log k) and streaming-friendly. Quickselect wins when k is a large fraction of d and you need all K but not their order. Sorting is fine only when d is small. Say that out loud, with the complexities, because the interviewer is grading the trade-off discussion.

FOLLOW-UP 4 The log is a stream and memory is limited.

Misra-Gries keeps m counters; any user with more than n / (m + 1) messages is guaranteed to remain in the summary (checked above). A second pass, or a count-min sketch, then gives the real counts of the survivors.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import heapq
from collections import Counter


def speaker(line):
    # "A says hi" -> "A"
    return line.split(" ", 1)[0]


def rank_key(item):
    # more messages first; ties by name
    return (-item[1], item[0])


# BASE QUESTION (optimal): Count, then keep a size-K heap
def top_k_speakers(chat_log, k):
    counts = Counter(speaker(line) for line in chat_log)
    # a heap of size k: O(n + d log k)
    items = counts.items()
    best = heapq.nsmallest(k, items, key=rank_key)
    return [user for user, _ in best]


# FOLLOW-UP 1: How do you do it across many machines when
# the log does not fit on ...
def merge_shards(shard_logs, k):
    # exact answer: add the full counts of every shard
    total = Counter()
    for log in shard_logs:
        total.update(speaker(line) for line in log)
    best = heapq.nsmallest(k, total.items(), key=rank_key)
    return [user for user, _ in best]


# FOLLOW-UP 2, 4: What if the log arrives as an unbounded
# stream?
class MisraGries:
    """m counters: a user with over n / (m + 1) of the
    n messages always stays in the summary."""

    def __init__(self, m):
        self.m, self.counts = m, {}

    def add(self, user):
        if user in self.counts:
            self.counts[user] += 1
        elif len(self.counts) < self.m:
            self.counts[user] = 1
        else:
            # no room: every counter pays one
            for u in list(self.counts):
                self.counts[u] -= 1
                if self.counts[u] == 0:
                    del self.counts[u]

    def candidates(self):
        return set(self.counts)


if __name__ == "__main__":
    log = ["A says hi", "B says hi", "A says bye",
           "C says hi", "A says yo"]
    assert top_k_speakers(log, 2) == ["A", "B"]
    assert top_k_speakers(log, 5) == ["A", "B", "C"]
    assert top_k_speakers([], 3) == []
    # two shards that each rank someone else first
    s1 = ["A says x", "B says x"]
    s2 = ["B says y", "B says z"]
    assert merge_shards([s1, s2], 1) == ["B"]
    mg = MisraGries(2)
    for u in "AAABBC":
        mg.add(u)
    # A has 3 of 6 messages (> 6 / 3), so it must survive
    assert "A" in mg.candidates()
```

NOTES FROM CANDIDATES

The interviewer typically wants a discussion of the three implementations + complexity trade-off, not just one working solution.
For the distributed follow-up, candidates that mention count-min sketch and topK per shard -> global merge got positive feedback.
The same family surfaces as "Top-K IPs", "Top-K chatters", and "Top-K words"; the technique is interchangeable.
A recent variant uses a gigabyte-scale IP list and explicitly requires exact time and space complexity analysis.

PREPARATION

Practice writing quickselect with the partition step (and explain how to recover from a bad pivot).
Be able to state amortized vs worst-case time for each approach.
Have a one-line answer for streaming: "hashmap counts + heap of size K maintained online".
Drill the distributed answer once: per-shard top-K, then K-way merge of the partial heaps.

STUDY HERE: topic 7, Heaps, Stacks, Sorting and Binary Search

Also fits: Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: binary-search, parallelism, concurrency, medium

MEDIUM | 1Point3Acres | phone-screen | 4 of 4

★ TOP 75: rank 72 in the Medium and Hard priority list

★ First Bad Version with Parallel Search Follow-up

ASKED 2026-06-15 | 127 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE **ALL TIME 41.1** **30 DAYS 33** **PRIORITY 47/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks, Sorting and Binary Search

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #278 First Bad Version

LeetCode match: LC 278 First Bad Version

Frequency: Single report | Last asked: 2026-06-15 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/first-bad-version-parallel-search

QUESTION

Phone-screen R1 coding round. The base problem is the standard first-bad-version monotonic search — given an `isBadVersion(v)` oracle over versions `1..n`, find the first bad version with a single binary search. The graded follow-up requires parallelizing the search: partition the range into buckets and search them concurrently (bucket + binary search), with working code expected.SWEbinary-searchparallelismconcurrencymediumFrequencySingle reportLast asked2026-06-15Stagephone-screen

APPROACH

Write the single-threaded binary search first and verify it on small n.
Then implement the bucketed parallel version with real concurrency primitives (Python concurrent.futures.ThreadPoolExecutor, or Java ExecutorService / CompletableFuture): split [1, n] into k buckets, probe the k boundaries concurrently, pick the bucket where the predicate flips, and binary-search within it. Practice getting this to compile and run in under 20 minutes.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what the oracle is (isBadVersion is monotone), what concurrency primitive to use (ThreadPoolExecutor, Java's ExecutorService), and which metric matters: wall-clock time or total calls.

BRUTE FORCE TO SAY OUT LOUD

Check every version from 1. O(n) calls.

INTERVIEW-DAY PLAN

OPTIMAL IDEA

Base: binary search, O(log n) calls. Parallel: each round fires up to k probes concurrently at evenly spaced points, finds the first probe that is bad, and narrows the interval to the gap between it and the previous probe. Each round shrinks the interval by k + 1, so there are about log base (k + 1) of n sequential rounds. Use a thread pool and map the probes in one call.

DRY-RUN THESE CASES

n = 1, the first version bad, the last version bad, n smaller than the number of workers.

LIKELY FOLLOW-UPS

Rate limiting, retries when a probe fails, and caching results.

PITFALLS

Mutating shared state from threads. Assuming more threads always means fewer total calls (they increase it).

HOW TO THINK ABOUT IT

'First bad version' is a boundary in a sorted yes/no sequence, which is binary search. The parallel part is a different trade: instead of one probe per round, send several probes in the same round and keep the gap that contains the boundary. Fewer rounds, more total calls.

Basis: LeetCode 278, First Bad Version, plus a concurrent k-ary search follow-up.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n):
    for v in range(1, n + 1):
        if is_bad_version(v):
            return v
```

TIME	SPACE
O(n)	O(1)
calls	

WHY IT IS SLOW

One call per version, O(n) calls; with a billion versions and a slow check that is far too many. The check is monotone, so binary search needs only log2(n) calls, and probing several points in parallel needs fewer rounds.

OPTIMAL SOLUTION · Base problem: binary search

TIME	SPACE
O(log n)	O(1)
calls	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Parallel: probe k points at once, then recurse into the bucket that flips

TIME	SPACE
O(log_{k+1} n) rounds	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What does parallelism buy you, and what does it cost?

It cuts the number of sequential rounds from log2(n) to log_{k+1}(n), so wall-clock time drops when each probe has high latency. It increases total probes (about k per round instead of 1), so it costs more calls and money. Say which metric you are optimizing before choosing k.

FOLLOW-UP 2 What if the oracle is rate limited or can fail?

Cap the pool size to the rate limit, retry transient failures with backoff (probes are idempotent), and never advance the interval from a probe that errored. Cache results by version so a retry or a later round never repeats a call.

FOLLOW-UP 3 Count how many rounds each approach needs.

Single binary search: log2(n) rounds of one probe each. k workers: about log base (k + 1) of n rounds with k probes each. For n = 10^9, one probe per round needs 30 rounds, while 7 probes per round need 10.

FOLLOW-UP 4 Show how many rounds and calls the parallel search uses.

With k workers each round shrinks the interval by about k + 1, so the number of sequential rounds is about log base (k + 1) of n and the total calls at most k times that. For n = 10^6 and 7 workers that is about 7 rounds and 49 calls, against 20 rounds and 20 calls for the sequential search: fewer rounds, more calls.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from concurrent.futures import ThreadPoolExecutor
from threading import Lock


# BASE QUESTION (optimal): problem: binary search
def first_bad_version(n, is_bad):
    # the base question: binary search on versions 1..n
    lo, hi = 1, n
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if is_bad(mid):
            # the first bad version is mid or earlier
            hi = mid
        else:
            lo = mid + 1
    return lo


# PARALLEL SEARCH helper
def probe_points(lo, hi, workers):
    # evenly spaced versions inside [lo, hi - 1]
    step = max(1, (hi - lo + 1) // (workers + 1))
    pts = {min(hi - 1, lo + step * (i + 1) - 1)
           for i in range(workers)}
    return sorted(pts)


# FOLLOW-UP 1, 3, 4: What does parallelism buy you, and
# what does it cost?
def first_bad_parallel(n, is_bad, workers=4):
    # invariant: the answer is in [lo, hi]
    lo, hi, cache, rounds = 1, n, {}, 0
    with ThreadPoolExecutor(workers) as pool:
        while lo < hi:
            points = probe_points(lo, hi, workers)
            # ask the oracle about new points at the same
            # time
            fresh = [p for p in points if p not in cache]
            answers = pool.map(is_bad, fresh)
            for p, bad in zip(fresh, answers):
                cache[p] = bad
            rounds += 1
            bad_points = [p for p in points if cache[p]]
            if not bad_points:
                # every probe was good: answer is after
                # them
                lo = points[-1] + 1
            else:
                first = bad_points[0]
                # the answer sits between the last good
                # probe before it and the first bad one
                before = [p for p in points if p < first]
                if before:
                    lo = before[-1] + 1
                hi = first
    return lo, rounds, len(cache)


if __name__ == "__main__":
    calls, lock = [], Lock()

    def oracle(first_bad):
        def is_bad(v):
            with lock:
                calls.append(v)
            return v >= first_bad
        return is_bad

    cases = ((1, 1), (10, 1), (10, 10), (1000, 637))
    for n, bad in cases:
        assert first_bad_version(n, oracle(bad)) == bad
        ans, rounds, used = first_bad_parallel(
            n, oracle(bad), workers=4)
        assert ans == bad
    # fewer rounds than the 20 of a plain binary search
    ans, rounds, used = first_bad_parallel(
        10 ** 6, oracle(123456), workers=7)
    assert ans == 123456 and rounds < 20
    assert used <= 7 * rounds
```

NOTES FROM CANDIDATES

The base is trivial; budget your time for the parallel follow-up and aim to get a complete, compiling version through even if it has minor bugs — finishing the parallel version is what passes this round.
Be explicit about what the parallel design optimizes: wall-clock latency (fire many isBadVersion probes at once) versus total number of calls. Splitting into k buckets and probing each boundary concurrently localizes the transition in one parallel round, then a binary search finishes it.
Common bugs: off-by-one when mapping a bucket index back to an absolute version range; the transition sitting exactly on a bucket boundary; integer overflow in mid if you write (lo + hi) / 2 instead of lo + (hi - lo) // 2.

PREPARATION

Write the single-threaded binary search first and verify it on small n.
Then implement the bucketed parallel version with real concurrency primitives (Python concurrent.futures.ThreadPoolExecutor, or Java ExecutorService / CompletableFuture): split [1, n] into k buckets, probe the k boundaries concurrently, pick the bucket where the predicate flips, and binary-search within it. Practice getting this to compile and run in under 20 minutes.